Assignment 3 Report

Joe Ferguson, Max Eichelberger

Contribution:

Joe did parts 1-3 & 6. Max did parts 4-5.

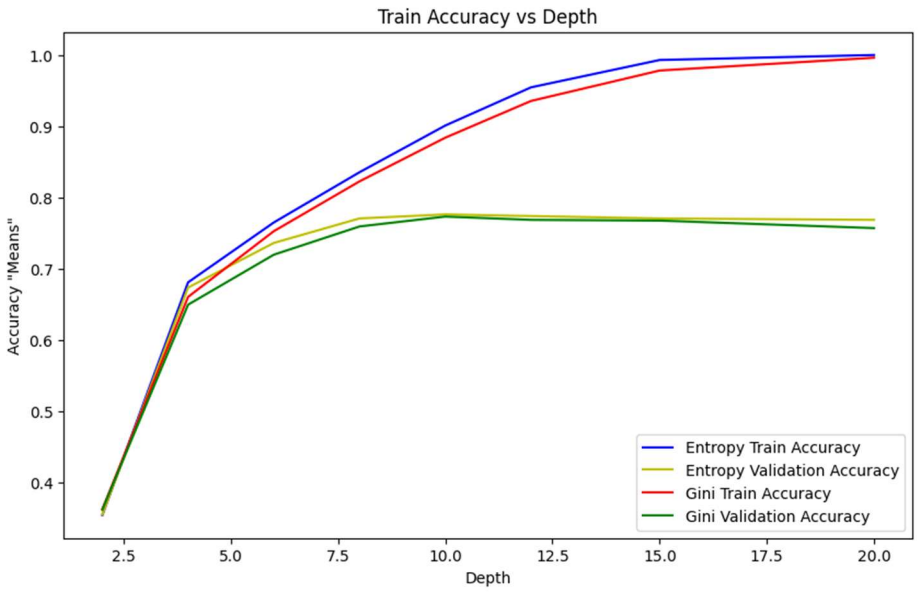
Notes:

To keep the report as concise as possible, no implementation details are added, only answers to the questions remain, I also decreased the size of some of the figures, so zooming in may be necessary. Running the entire jupyter notebook will take over 3 hrs due to the length of the assignment, only run what is necessary. **All of the figures have been combined into the zip file if closer inspection is needed.**

Part 1: Single CART Trees

1a

1b



Entropy Criterion Results

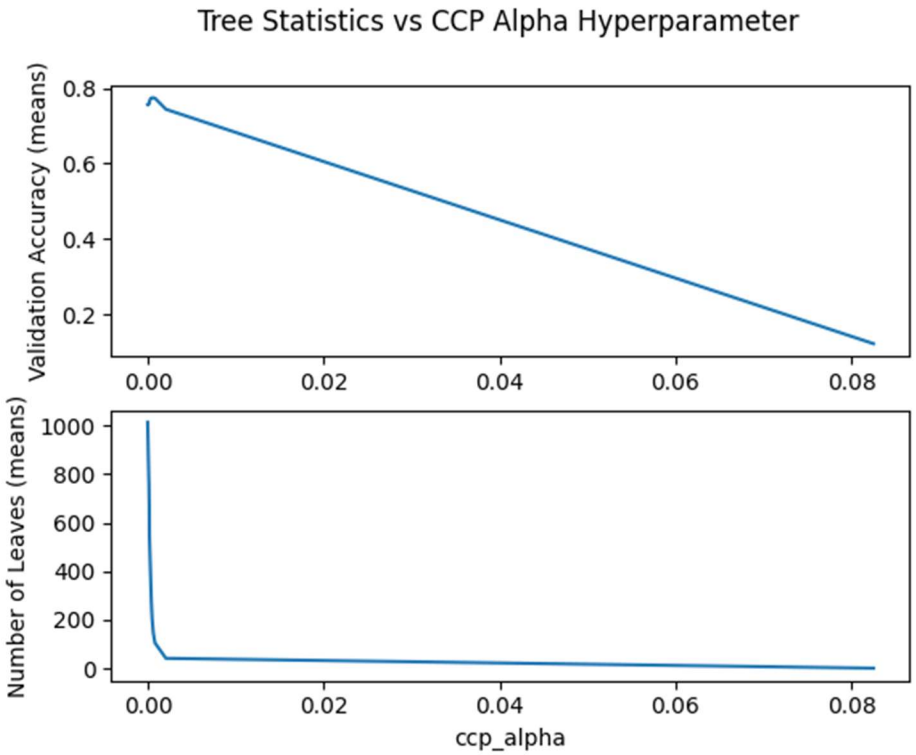
Depth	Entropy Train Accuracy	Entropy Validation Accuracy	Entropy Leaves	Entropy Actual Depth	Entropy Times
2.0	0.354 ± 0.001	0.355 ± 0.002	4.000 ± 0.000	2.000 ± 0.000	0.492 ± 0.016
4.0	0.681 ± 0.004	0.674 ± 0.004	16.000 ± 0.000	4.000 ± 0.000	1.035 ± 0.018
6.0	0.765 ± 0.006	0.736 ± 0.006	62.400 ± 0.548	6.000 ± 0.000	1.678 ± 0.029
8.0	0.835 ± 0.004	0.771 ± 0.007	189.000 ± 4.301	8.000 ± 0.000	2.293 ± 0.030
10.0	0.901 ± 0.005	0.776 ± 0.005	389.800 ± 9.731	10.000 ± 0.000	2.841 ± 0.068
12.0	0.955 ± 0.004	0.774 ± 0.006	624.000 ± 10.173	12.000 ± 0.000	3.192 ± 0.068
15.0	0.993 ± 0.002	0.771 ± 0.006	812.400 ± 7.092	15.000 ± 0.000	3.374 ± 0.041
20.0	1.000 ± 0.000	0.769 ± 0.006	848.000 ± 6.164	19.600 ± 0.548	3.363 ± 0.040
nan	1.000 ± 0.000	0.768 ± 0.006	848.400 ± 5.639	19.800 ± 0.837	3.404 ± 0.075

Depth	Gini Train Accuracy	Gini Validation Accuracy	Gini Leaves	Gini Actual Depth	Gini Times
2.0	0.362 ± 0.001	0.362 ± 0.003	4.000 ± 0.000	2.000 ± 0.000	0.421 ± 0.009
4.0	0.660 ± 0.002	0.650 ± 0.007	16.000 ± 0.000	4.000 ± 0.000	0.805 ± 0.019
6.0	0.753 ± 0.007	0.720 ± 0.010	59.200 ± 0.837	6.000 ± 0.000	1.209 ± 0.029
8.0	0.823 ± 0.003	0.759 ± 0.009	161.800 ± 4.324	8.000 ± 0.000	1.660 ± 0.068
10.0	0.884 ± 0.001	0.773 ± 0.007	325.000 ± 6.403	10.000 ± 0.000	2.001 ± 0.069
12.0	0.935 ± 0.003	0.769 ± 0.008	543.800 ± 11.883	12.000 ± 0.000	2.311 ± 0.058
15.0	0.978 ± 0.002	0.768 ± 0.006	809.600 ± 26.121	15.000 ± 0.000	2.702 ± 0.119
20.0	0.996 ± 0.001	0.757 ± 0.006	966.600 ± 15.662	20.000 ± 0.000	3.058 ± 0.112
nan	1.000 ± 0.000	0.756 ± 0.006	1014.000 ± 10.583	31.600 ± 3.209	3.257 ± 0.118

```
Root agreement: 0.000 ± 0.000
Depth 1 agreement: 0.000 ± 0.000
Depth 2 agreement: 0.050 ± 0.112
Total agreement: 0.017 ± 0.065
```

Looking at the graphics, we can say the criteria matter very little compared to the depth the trees are allowed. We also see that because of the different metrics, the trees do not seem to agree with which feature should be in depths 0-2.

1c

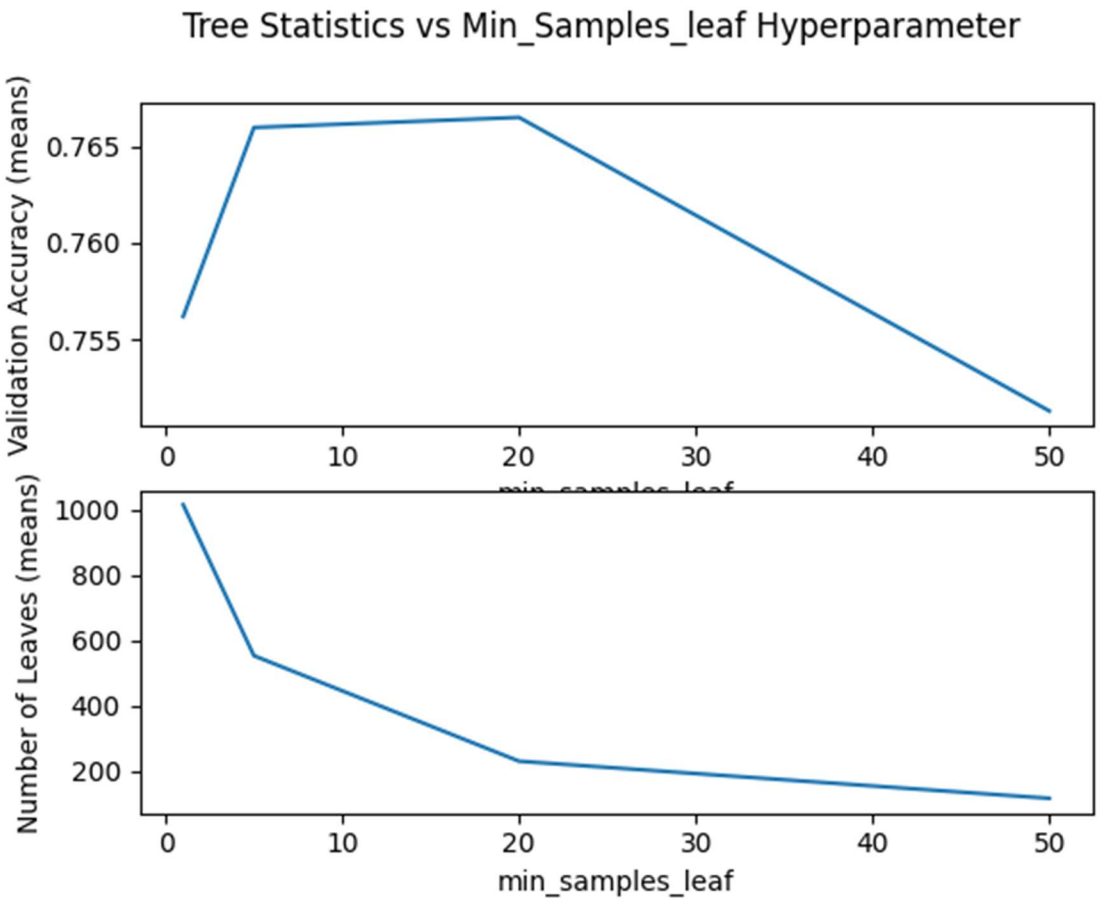



```
Optimal Alpha: 0.0006
Best Pruning Validation Accuracy 0.7672
```

```
Best Depth-Limited Tree Validation Accuracy : 0.7548
```

As we can see the optimal alpha for CCP beats the best depth-limited tree on the test sets, this is due to the pruning, which reduces the amount the tree will overfit, thus making it generalize better on unseen data.

Trying min_samples_leaf, we get:




```
Optimal min_samples_leaf: 5.0000
Best min_samples_leaf Validation Accuracy 0.7636
```

As we can see the optimal min_samples_leaf also beats the best depth-limited tree, for the same reason. Although it looks like 20 should be the optimal value based on the chart, GridSearch determined 5 was the optimal value for random seed 1.

For both optimal values, I did not average the values over 5 runs as GridSearch is very compute intensive and would take my labtop ~ 1 hour to run.

However, the charts show the average means.

1d

3 Regression Metrics - Evaluated using RMSE

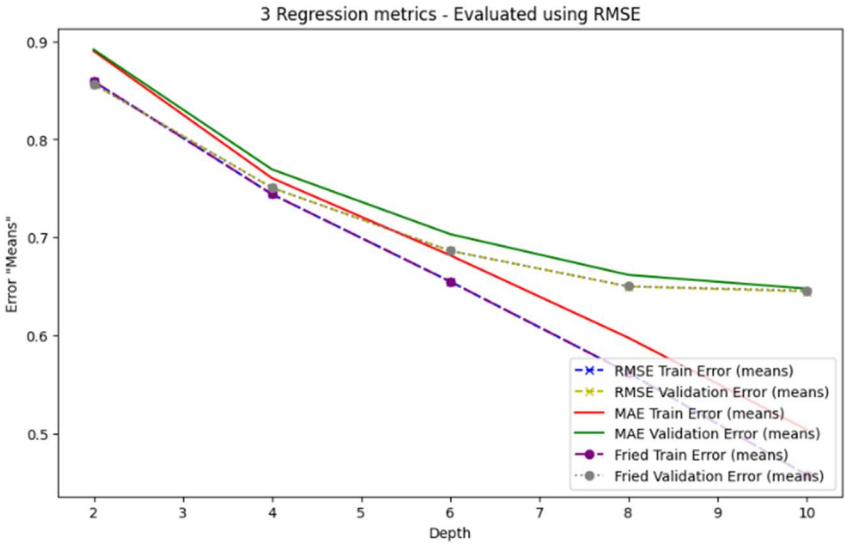
Depth	RMSE Train Error - Eval RMSE	MAE Train Error - Eval RMSE	Friedman Train Error - Eval RMSE	RMSE Validation Error - Eval RMSE	MAE Validation Error - Eval RMSE	Friedman Validation Error - Eval RMSE
2	0.859 ± 0.002	0.890 ± 0.003	0.859 ± 0.002	0.856 ± 0.010	0.892 ± 0.008	0.856 ± 0.010
4	0.744 ± 0.003	0.761 ± 0.003	0.744 ± 0.003	0.751 ± 0.010	0.770 ± 0.007	0.751 ± 0.010
6	0.655 ± 0.008	0.682 ± 0.006	0.655 ± 0.008	0.686 ± 0.008	0.703 ± 0.011	0.686 ± 0.008
8	0.562 ± 0.010	0.598 ± 0.006	0.562 ± 0.010	0.650 ± 0.016	0.662 ± 0.009	0.650 ± 0.016
10	0.458 ± 0.010	0.504 ± 0.006	0.458 ± 0.010	0.645 ± 0.014	0.648 ± 0.008	0.646 ± 0.013

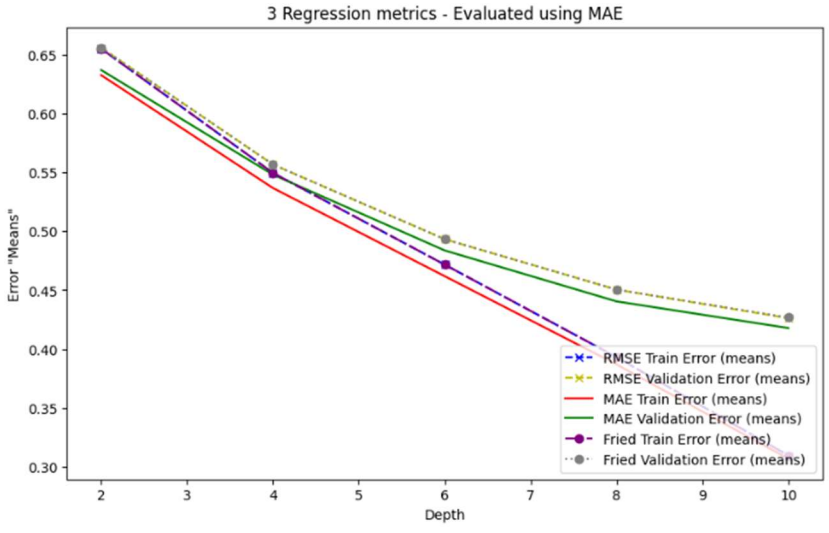
3 Regression Metrics - Evaluated using MAE

Depth	RMSE Train Error - Eval MAE	MAE Train Error - Eval MAE	Friedman Train Error - Eval MAE	RMSE Validation Error - Eval MAE	MAE Validation Error - Eval MAE	Friedman Validation Error - Eval MAE
2	0.655 ± 0.002	0.632 ± 0.003	0.655 ± 0.002	0.656 ± 0.007	0.637 ± 0.007	0.656 ± 0.007
4	0.549 ± 0.002	0.537 ± 0.002	0.549 ± 0.002	0.557 ± 0.005	0.548 ± 0.006	0.557 ± 0.005
6	0.472 ± 0.007	0.462 ± 0.003	0.472 ± 0.007	0.493 ± 0.005	0.484 ± 0.007	0.493 ± 0.005
8	0.393 ± 0.009	0.387 ± 0.006	0.393 ± 0.009	0.450 ± 0.013	0.440 ± 0.010	0.451 ± 0.013
10	0.310 ± 0.007	0.307 ± 0.005	0.310 ± 0.007	0.426 ± 0.009	0.418 ± 0.007	0.427 ± 0.009

3 Regression Metrics - Evaluated using Friedman MSE

Depth	RMSE Train Error - Eval MSE	MAE Train Error - Eval MSE	Friedman Train Error - Eval MSE	RMSE Validation Error - Eval MSE	MAE Validation Error - Eval MSE	Friedman Validation Error - Eval MSE
2	0.738 ± 0.003	0.793 ± 0.006	0.738 ± 0.003	0.734 ± 0.017	0.795 ± 0.015	0.734 ± 0.017
4	0.554 ± 0.004	0.578 ± 0.005	0.554 ± 0.004	0.564 ± 0.015	0.592 ± 0.010	0.564 ± 0.015
6	0.429 ± 0.011	0.465 ± 0.008	0.429 ± 0.011	0.471 ± 0.012	0.495 ± 0.016	0.471 ± 0.012
8	0.316 ± 0.011	0.357 ± 0.008	0.316 ± 0.011	0.423 ± 0.021	0.438 ± 0.012	0.423 ± 0.021
10	0.209 ± 0.009	0.254 ± 0.006	0.209 ± 0.009	0.416 ± 0.018	0.420 ± 0.010	0.417 ± 0.017

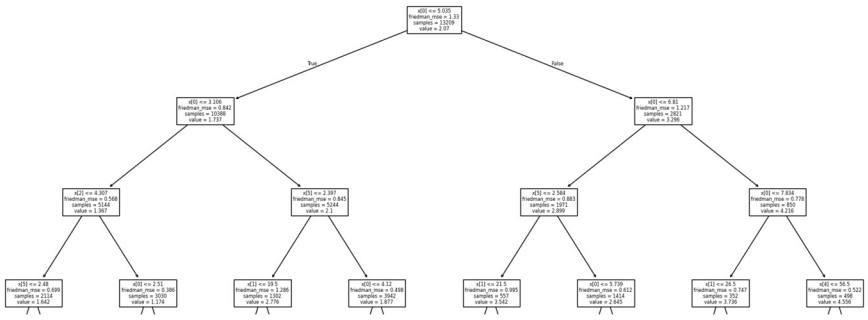




```
Best Validation RMSE: 0.624
```

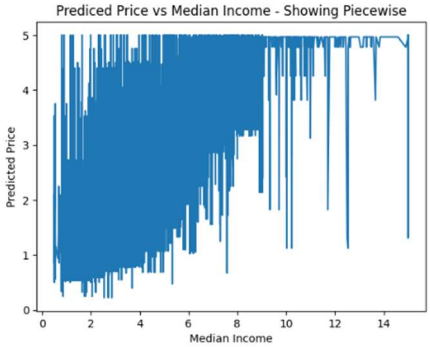
As we can see Friedman mse and RMSE are equivalent as they are mathematical proportional to each other, which can result in identical trees, for the trees trained on one metric, they are better when evaluated on that same metric then for another one. This is to be expected logically, as they are minimizing that metric through training.

Top 3 Levels Of Tree



```
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
```

The root and depth-1 levels are splitting based on MedInc, which means it has the most feature importance and purity. Further down, HouseAge, AveRooms, AveOccup, and Population start to be considered. This is to be expected, we should not be seeing the Latitude or Longitude near the top of the tree.



This shows the step-function (piecewise) behavior of the regression tree. Although cluttered, we can see the steps “up the staircase” when median income increases.

Part 2: Sampling-based ensembles

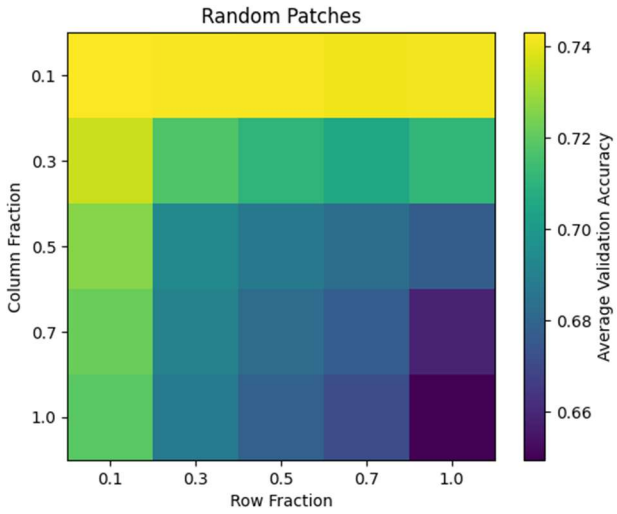
2a

Sampling Based Ensembles Train Accuracy

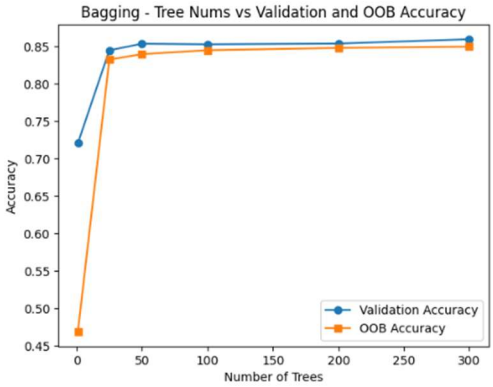
Fractions	Pasting Train Accuracy	Bagging Train Accuracy	Random Subspaces Train Accuracy	Random Patches Train Accuracy (Averaged by Row)	Random Patches Train Accuracy (Averaged by Column)
0.1	0.727 ± 0.019	0.734 ± 0.012	0.748 ± 0.003	0.736 ± 0.008	0.742 ± 0.003
0.3	0.697 ± 0.019	0.708 ± 0.023	0.720 ± 0.014	0.715 ± 0.013	0.716 ± 0.010
0.5	0.688 ± 0.021	0.698 ± 0.019	0.687 ± 0.021	0.709 ± 0.015	0.693 ± 0.017
0.7	0.682 ± 0.020	0.693 ± 0.021	0.670 ± 0.002	0.705 ± 0.015	0.687 ± 0.015
1.0	0.660 ± 0.007	0.690 ± 0.020	0.660 ± 0.007	0.697 ± 0.007	0.681 ± 0.014

Sampling Based Ensembles Validation Accuracy

Fractions	Pasting Validation Accuracy	Bagging Validation Accuracy	Random Subspaces Validation Accuracy	Random Patches Validation Accuracy (Averaged by Row)	Random Patches Validation Accuracy (Averaged by Column)
0.1	0.719 ± 0.016	0.722 ± 0.007	0.741 ± 0.005	0.729 ± 0.006	0.742 ± 0.003
0.3	0.688 ± 0.021	0.699 ± 0.024	0.712 ± 0.012	0.707 ± 0.014	0.716 ± 0.010
0.5	0.679 ± 0.022	0.689 ± 0.021	0.677 ± 0.022	0.700 ± 0.015	0.693 ± 0.017
0.7	0.672 ± 0.020	0.683 ± 0.022	0.659 ± 0.006	0.696 ± 0.015	0.687 ± 0.015
1.0	0.650 ± 0.007	0.680 ± 0.021	0.650 ± 0.007	0.688 ± 0.007	0.681 ± 0.014

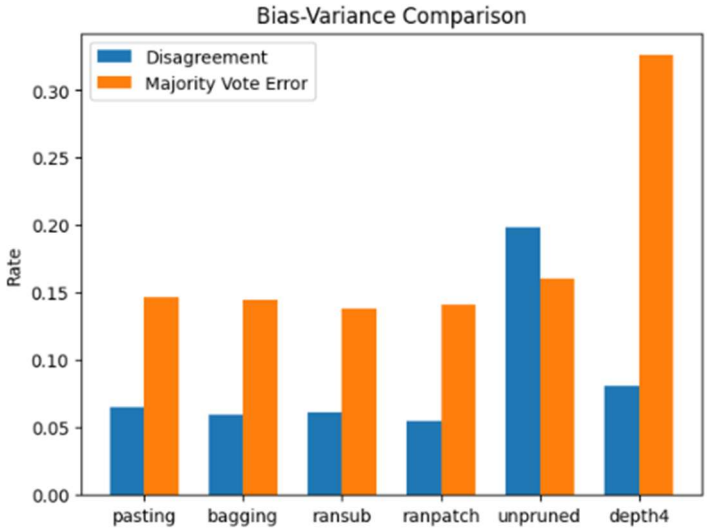


2b



The validation accuracy and OOB estimate are very close, both estimate how well the model generalizes to unseen data. Pasting does not have one because it samples, without replacement, so it will use every sample from the subset.

2c



As we can see the bias remains the same vs an unpruned tree, but the averaging reduces disagreement.

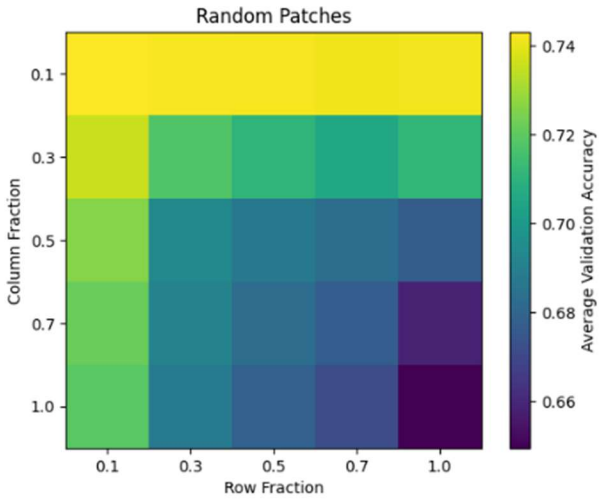
2d

Sampling Based Ensembles Training Stats

Fraction	Pasting Train Accuracy	Bagging Train Accuracy	Pasting Subspaces Train Accuracy	Random Patches Train Accuracy (Averaged by Row)	Random Patches Train Accuracy (Averaged by Column)
0.1	0.727 ± 0.019	0.734 ± 0.012	0.748 ± 0.008	0.758 ± 0.008	0.742 ± 0.008
0.3	0.687 ± 0.019	0.703 ± 0.023	0.729 ± 0.017	0.725 ± 0.013	0.716 ± 0.015
0.5	0.668 ± 0.021	0.684 ± 0.019	0.687 ± 0.021	0.703 ± 0.015	0.691 ± 0.013
0.7	0.662 ± 0.020	0.663 ± 0.021	0.673 ± 0.004	0.695 ± 0.015	0.687 ± 0.015
1.0	0.660 ± 0.002	0.659 ± 0.020	0.663 ± 0.002	0.697 ± 0.007	0.681 ± 0.014

Sampling Based Ensembles Training Stats

Fraction	Pasting Validation Accuracy	Bagging Validation Accuracy	Pasting Subspaces Validation Accuracy	Random Patches Validation Accuracy (Averaged by Row)	Random Patches Validation Accuracy (Averaged by Column)
0.1	0.726 ± 0.018	0.722 ± 0.007	0.741 ± 0.005	0.723 ± 0.006	0.742 ± 0.008
0.3	0.686 ± 0.021	0.699 ± 0.024	0.722 ± 0.012	0.707 ± 0.013	0.715 ± 0.018
0.5	0.670 ± 0.017	0.680 ± 0.021	0.677 ± 0.022	0.700 ± 0.015	0.695 ± 0.017
0.7	0.672 ± 0.020	0.683 ± 0.022	0.679 ± 0.009	0.699 ± 0.015	0.687 ± 0.015
1.0	0.659 ± 0.020	0.659 ± 0.021	0.663 ± 0.002	0.698 ± 0.007	0.681 ± 0.014



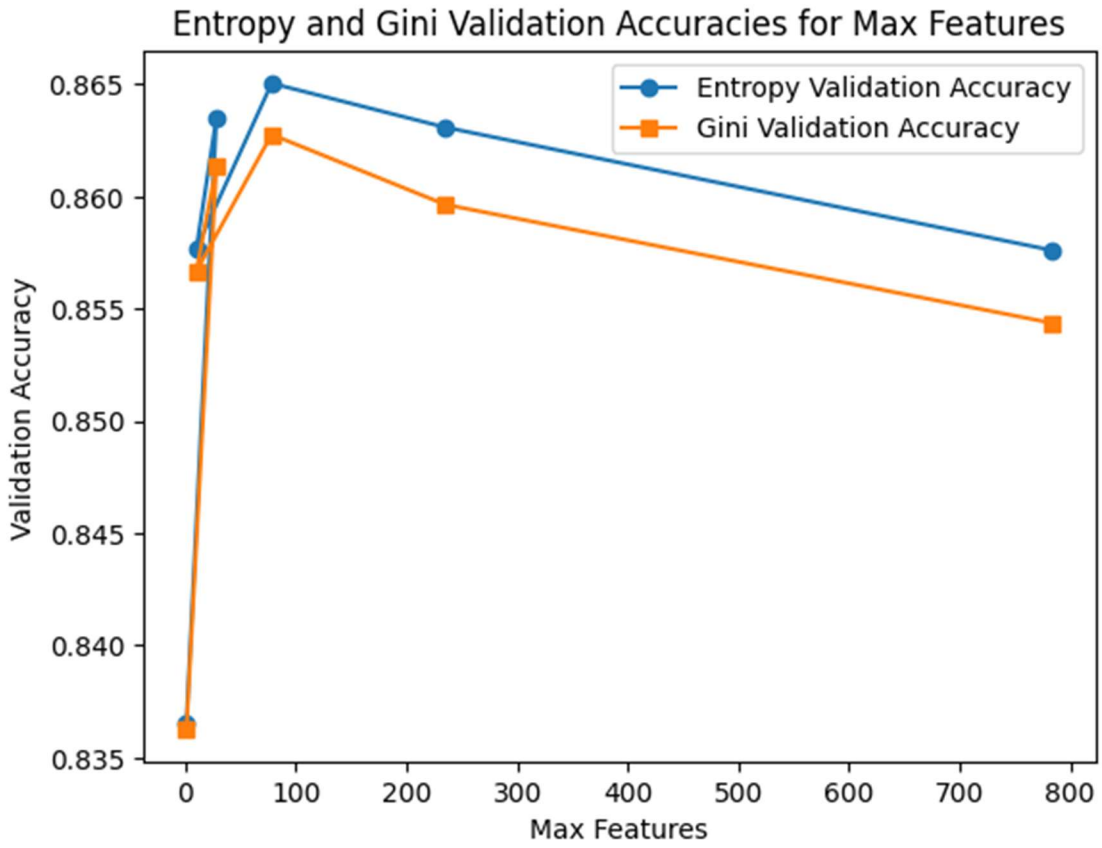
Shallow trees have high bias but lower variance, i.e they underfit, so bagging does little since it reduces variance. However, for deep trees, which have higher bias, bagging can help reduce the variance which improves their performance a lot more.

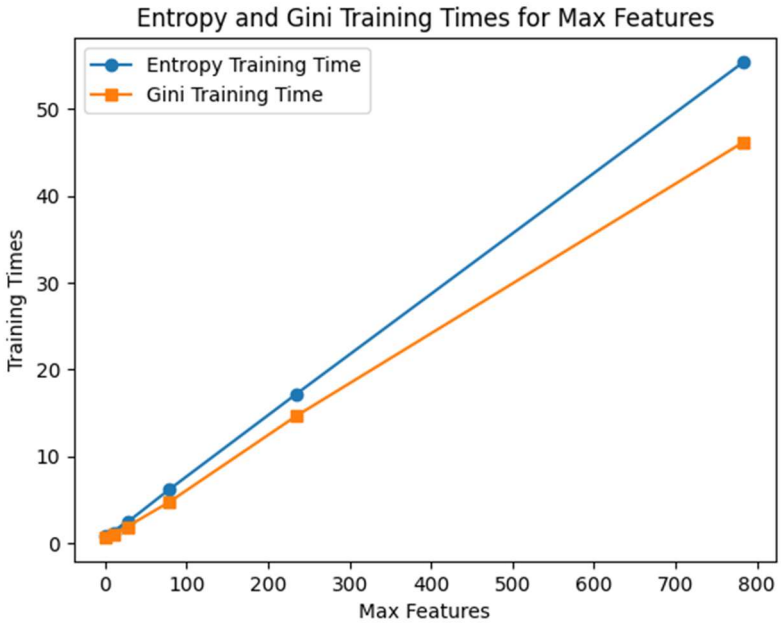
Part 3 – Random Forests and Extra Trees

3a

Random Forest Entropy vs Gini Stats

Max Features	Random Forest Entropy Validation Accuracy	Random Forest Gini Validation Accuracy	Random Forest Entropy Training Times	Random Forest Gini Training Times
1	0.827 ± 0.009	0.836 ± 0.006	0.786 ± 0.057	0.647 ± 0.055
28	0.861 ± 0.005	0.861 ± 0.004	2.488 ± 0.101	1.822 ± 0.075
50	0.858 ± 0.004	0.857 ± 0.003	1.115 ± 0.040	0.942 ± 0.057
78	0.865 ± 0.003	0.865 ± 0.003	6.110 ± 0.135	4.672 ± 0.059
235	0.863 ± 0.003	0.860 ± 0.003	17.193 ± 0.104	14.662 ± 0.659
784	0.858 ± 0.004	0.854 ± 0.003	55.472 ± 1.197	46.237 ± 1.225



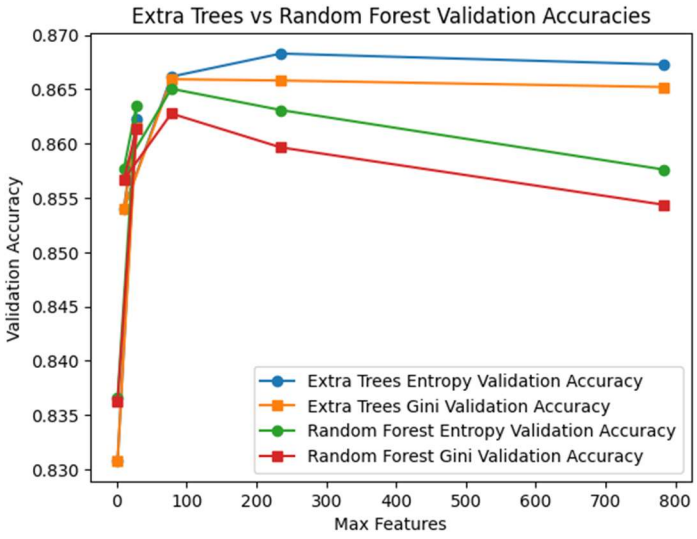


As we can see the initial values of Max Features result with an increase in validation accuracies which diminish as the hyperparameter increase. Also to be noted is that entropy does result in slightly better accuracy at the cost of slightly longer training times.

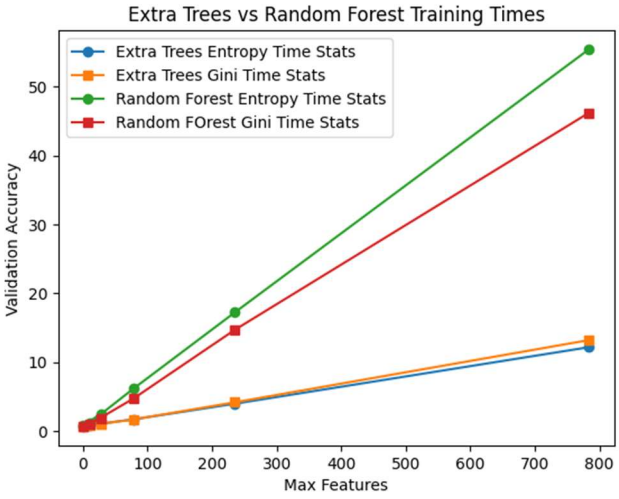
3b

Extra Trees Entropy vs Gini Stats

Max Features	Extra Trees Entropy Validation Accuracy	Extra Trees Gini Validation Accuracy	Extra Trees Entropy Training Times	Extra Trees Gini Training Times
1	0.831 ± 0.008	0.831 ± 0.008	0.657 ± 0.021	0.632 ± 0.070
28	0.862 ± 0.004	0.861 ± 0.005	1.031 ± 0.016	1.014 ± 0.029
10	0.854 ± 0.007	0.854 ± 0.005	0.805 ± 0.046	0.793 ± 0.049
78	0.866 ± 0.004	0.866 ± 0.005	1.654 ± 0.020	1.636 ± 0.031
235	0.868 ± 0.005	0.866 ± 0.004	3.926 ± 0.074	4.130 ± 0.057
784	0.867 ± 0.004	0.865 ± 0.004	12.150 ± 0.518	13.180 ± 0.247

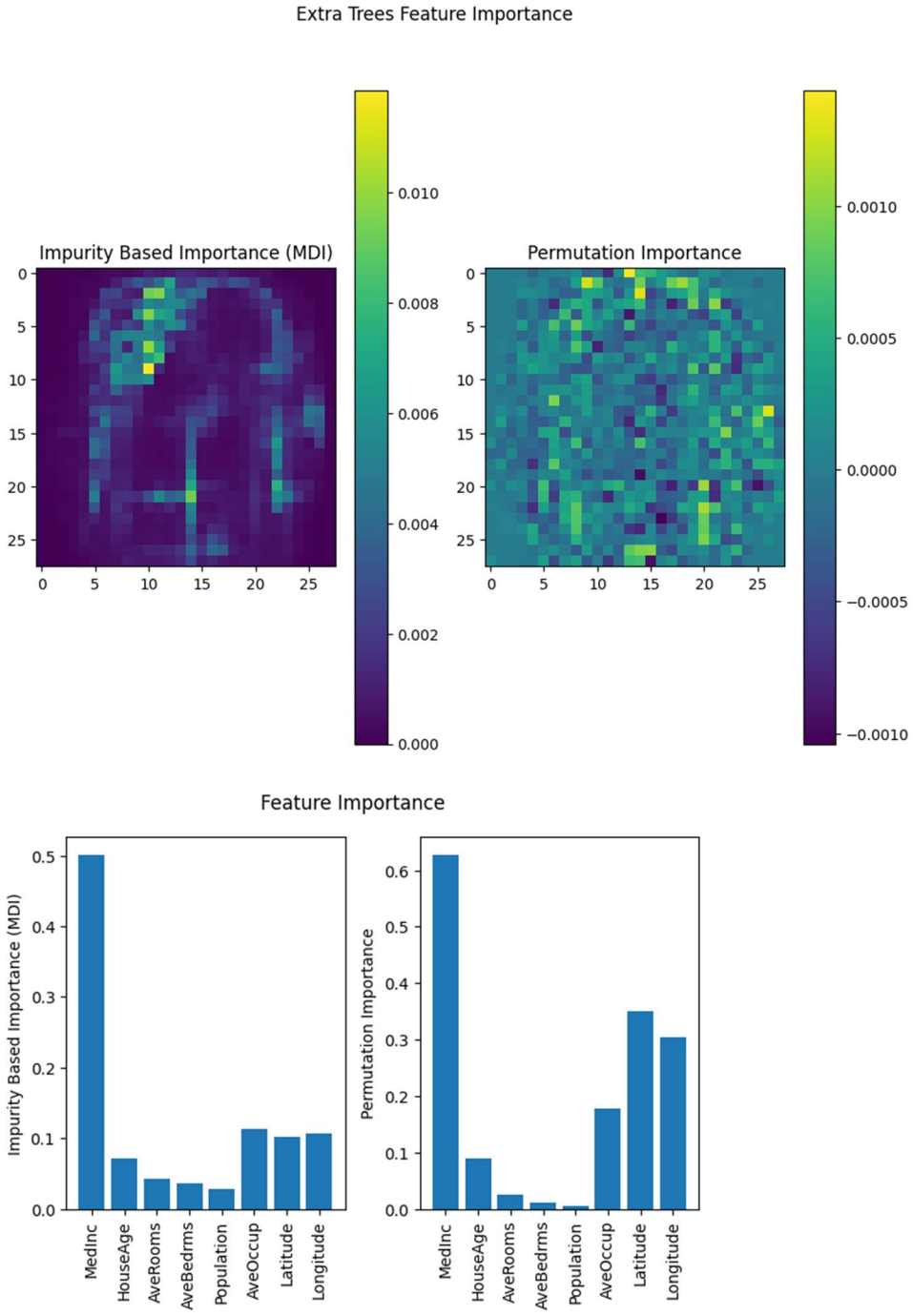


The extra trees perform better.



The extra trees perform better across both accuracy and training time, with similar variance if we compare the table in 3b to 3a. This could be data-dependent, both do similar jobs of reducing variance.

3c



The difference between MDI and Permutation importance is that MDI measures total reduction in impurity for a feature, while the other measures the increase in prediction error when a feature's values are randomly shuffled. Impurity importance can be biased towards high-cardinality features since they allow more splitting thresholds (more distinct values), which increases the chance of having a large impurity reduction.

Part 4 – Boosting

Part 5 – Combining Classifiers

Part 6 – Final Comparison and Report

Classification Report

Method	Best Hyperparameters	Test Accuracy (%)	Macro-F1	Train Time (s)	Predict Time (ms / 1k images)	Model Size (nodes)
Tree (Gini)	ccp_alpha=0.0006, min_samples_leaf=20, max_depth=None	0.763 ± 0.007	0.763 ± 0.007	2.051 ± 0.035	3.639 ± 1.230	229.000 ± 8.672
Tree (Entropy)	ccp_alpha=0.0006, min_samples_leaf=20, max_depth=None	0.767 ± 0.005	0.767 ± 0.005	2.564 ± 0.032	2.930 ± 0.069	437.400 ± 9.749
Pasting	n_estimators=300, max_samples=0.1, max_features=1.0, bootstrap=False	0.821 ± 0.002	0.818 ± 0.002	7.223 ± 3.642	227.304 ± 6.049	81399.200 ± 504.433
Bagging	n_estimators=300, max_samples=1.0, max_features=0.1, bootstrap=True	0.858 ± 0.001	0.857 ± 0.001	7.269 ± 0.065	87.020 ± 5.090	822632.800 ± 4200.731
Random Subspaces	n_estimators=300, max_samples=1.0, max_features=0.1, bootstrap=False	0.858 ± 0.001	0.857 ± 0.001	7.251 ± 0.120	87.384 ± 3.561	822632.800 ± 4200.731
Random Patches	n_estimators=300, max_samples=0.1, max_features=0.1, bootstrap=False	0.824 ± 0.002	0.821 ± 0.002	1.028 ± 0.084	72.979 ± 4.533	107926.400 ± 627.585
Random Forest	n_estimators=300, criterion=entropy, max_features=100, bootstrap=True	0.857 ± 0.001	0.856 ± 0.001	6.854 ± 0.079	12.908 ± 0.906	446716.400 ± 2152.034
Extra Trees	n_estimators=300, criterion=entropy, max_features=100	0.859 ± 0.001	0.858 ± 0.001	2.021 ± 0.088	17.712 ± 2.317	960364.400 ± 5639.100
AdaBoost	max_depth=8, n_estimators=339, learning_rate=0.1	0.861 ± 0.004	0.862 ± 0.004	588.275 ± 1.576	496.866 ± 18.192	126151.000 ± 1088.691
Gradient Boosting	max_depth=4, max_iter=500, learning_rate=0.1, early_stopping=True	0.865 ± 0.002	0.865 ± 0.001	51.845 ± 2.984	296.748 ± 18.882	36454.400 ± 2228.956
Hard voting	DecisionTree, RandomForest, AdaBoost, HistGradientBoost All best hyperparameters above	0.863 ± 0.001	0.862 ± 0.001	679.258 ± 1.835	891.457 ± 58.272	768969.200 ± 5155.788
Soft voting	DecisionTree, RandomForest, AdaBoost, HistGradientBoost All best hyperparameters above	0.857 ± 0.001	0.856 ± 0.001	679.258 ± 1.835	890.582 ± 58.266	768969.200 ± 5155.788
Stacking (LogReg)	DecisionTree, RandomForest, AdaBoost, HistGradientBoost All best hyperparameters above Max_iter=2000	0.867 ± 0.001	0.867 ± 0.001	683.972 ± 2.104	890.635 ± 58.272	768969.200 ± 5155.788

Regression Report

Method	Best Hyperparameters	Test RMSE	Test MAE	Train Time (s)	Predict Time (ms / 1k samples)	Model Size (nodes)
Tree (RMSE)	criterion=squared_error, ccp_alpha=0.0006, min_samples_leaf=20	0.625 ± 0.011	0.432 ± 0.007	0.061 ± 0.002	0.232 ± 0.027	269.800 ± 4.833
Tree (MAE)	criterion=absolute_error, ccp_alpha=0.0006, min_samples_leaf=20	0.639 ± 0.006	0.419 ± 0.004	4.275 ± 0.042	0.214 ± 0.008	191.800 ± 13.363
Pasting	n_estimators=300, max_samples=0.1, max_features=1.0, bootstrap=False	0.558 ± 0.007	0.383 ± 0.004	0.270 ± 0.002	35.559 ± 2.466	763419.600 ± 369.420
Bagging	n_estimators=300, max_samples=1.0, max_features=1.0, bootstrap=True	0.511 ± 0.010	0.334 ± 0.006	1.637 ± 0.017	168.989 ± 9.522	4829283.200 ± 3444.103
Random Subspaces	n_estimators=300, max_samples=1.0, max_features=0.5, bootstrap=False	0.548 ± 0.007	0.397 ± 0.004	1.852 ± 0.008	296.373 ± 49.090	7695164.400 ± 5582.213
Random Patches	n_estimators=300, max_samples=0.5, max_features=0.5, bootstrap=False	0.553 ± 0.006	0.403 ± 0.005	0.886 ± 0.016	160.698 ± 5.564	3850067.600 ± 2308.449
Random Forest	n_estimators=300, criterion=absolute_error, max_features=all available (8), bootstrap=True	0.505 ± 0.008	0.331 ± 0.005	43.923 ± 0.635	13.992 ± 1.006	4776740.000 ± 3825.480
Extra Trees	n_estimators=300, criterion=absolute_error, max_features=all available (8), bootstrap=True	0.506 ± 0.007	0.335 ± 0.005	27.034 ± 0.500	14.456 ± 0.090	4751161.200 ± 3494.765
AdaBoost	depth=8, rounds=63, lr=0.5, loss=square	0.545 ± 0.007	0.416 ± 0.008	2.063 ± 0.027	5.377 ± 0.454	23914.200 ± 739.602
Gradient Boosting	depth=6, n_estimators=2000, lr=0.03, early-stopping	0.466 ± 0.009	0.307 ± 0.006	24.730 ± 3.523	67816.800 ± 9339.743	67816.800 ± 9339.743
Mean voting	DecisionTree, RandomForest, AdaBoost, HistGradientBoost All best hyperparameters above	0.500 ± 0.008	0.345 ± 0.005	638.713 ± 7.401	4868740.800 ± 8294.180	4868740.800 ± 8294.180
Stacking (Ridge)	DecisionTree, RandomForest, AdaBoost, GradientBoost	0.464 ± 0.009	0.304 ± 0.006	638.948 ± 7.400	4868740.800 ± 8294.180	4868740.800 ± 8294.180

	All best hyperparameters above and Ridge where alpha=1.0					

Conclusions:

The baseline single trees reach 76% classification accuracy and 0.63 RMSE with little tuning. The sampling methods increase this to 83~86% and 0.51~0.56, which is already a good result for this dataset. The boosting methods increase this further to 86-87% and 0.47~0.55, which is only a slight improvement for the drastic increase in compute required. Finally the voting methods, which are the most complex and compute heavy, which barely tune the results further to 86~87% and 0.47~0.5. This could be due to underfitting from the underlying methods, there is no further improvement from reducing variance as the models are not complex enough for the details in the complex data.

Upon researching benchmarks for these datasets, the results are good for models that do not fall into more complex deep-learning methods.

Confusion Matrix for **Stacking**

```
[[787   0  11  34   3   1 156   0   8   0]
 [  3 963   4  22   3   0   5   0   0   0]
 [  8   0 791   7 111   0  81   0   2   0]
 [ 21   3  16 876  33   0  49   0   2   0]
 [  2   0 100  32 805   0  61   0   0   0]
 [  0   0   0   1   0 950   0  33   0  16]
 [120   1 109  26  87   0 645   0  12   0]
 [  0   0   0   0   0  18   0 938   0  44]
 [  0   2   5   2   4   2  14   5 966   0]
 [  0   0   0   1   0   5   0  41   1 952]]
```

The best method for

a) Accuracy is the **stacking** model using DecisionTree, RandomForest, AdaBoost, and Gradient Boosting
b) Tight Compute Budget is the **random forest or extra trees** model, where neither are clearly better than the other for this case. However, both perform near the same as the way more compute-heavy voting/stacking models, making them a solid pick.
c) For when the model must be explained to a non-expert, **bagging**. It is quite easy to explain using subsets with replacement of decision tree classifiers, and how this brings down variance for the final estimate. Compare this with the boosting and voting methods, which are more involved.